Movie Memory - submission summary

Balakrishna Nair Nalivalapil

Repository: github.com/bkrishnanair/Movie-Memory

Scope and choice

Implemented the base application and **Variant A: backend caching and correctness**. This choice focuses on ownership, concurrency, and failure handling, while keeping the interface small and building on my backend experience.

What is implemented

- Google sign-in with database sessions, protected pages, and logout.
- First-time onboarding with server-side movie validation and session-based ownership.
- Dashboard with profile fallbacks, saved movie, fact loading, refresh, and retry.
- Server-only OpenAI integration, with optional Gemini for local use; facts are saved in Postgres.
- A per-user/movie cache for facts younger than 60 seconds. Overlapping requests share one generation within a Node process. Failures return the last saved fact, or a friendly error when no fact exists.

Design and limitations

One Next.js/React/TypeScript application uses Tailwind, Prisma, and Postgres. User, Account, Session, and Fact records separate profile, login, and fact history. The server gets ownership from the verified session. Provider requests have a 10-second timeout and no automatic retries. Secrets stay on the server.

The in-progress guard supports one process only. Multiple instances need database-backed coordination. Movie titles are free text, and generated facts are not independently fact-checked. Variant B and deployment are outside this submission's scope.

Verification and setup

68 unit tests pass, along with lint, type checks, and a production build. Both migrations and HTTP checks passed against Postgres, including authorization, cache boundaries, concurrency, stale fallback, and retry with a local provider fixture.

Google sign-in, onboarding, saving a movie, logout, protected routes, and returning-user routing passed browser checks. Gemini generated a real fact that was saved and reused. OpenAI remains the default required integration, but its live check was blocked by exhausted credit. The local demo uses Gemini; this does not establish successful live OpenAI generation.

The README contains environment variables, Google callback configuration, migrations, architecture, and repeatable checks. Start with Node 24, Postgres 16+, npm ci, a configured .env, npm run db:deploy, and npm run dev.

Next improvements and tool use

With two more hours: add a browser regression for onboarding/return/logout and improve ambiguous-title feedback. Before scaling, add database-backed generation coordination.

- Codex helped with planning, implementation, tests, debugging, and documentation.
- Verification results and remaining live-service gaps are stated above.